



MIT Center for
Transportation &
Logistics



Omnichannel Supply Chain Lab



Advanced Warehouse Simulation Tool Demo

Shailesh Divey¹

¹MIT Center for Transportation & Logistics

May 1, 2025

Conceptual Model & Approach

High-Level Flow

1. Orders Arrive (Poisson process)
2. Resource Requests (crane, shuttle, robot)
3. Processing Steps (travel times, picking times)
4. Cost & Throughput Calculation
5. Key Assumptions
 - a. Poisson arrivals (random inter-arrival times).
 - b. Travel times = distance / speed.
 - c. Single or double depth, single or dual command cycles as multipliers.
 - d. Cross-docking → bypass retrieval if chosen.

Parameters & Inputs

1. Arrivals (Poisson)

Malmborg (2003); Fukunari & Malmborg (2009); Heragu, Cai, Krishnamurthy, Malmborg (2011); Marchet, Melacini, Perotti, Tappia (2013).

2. Resources (Cranes, shuttles, robots)

Gagliardi, Renaud, Luiz (2013); Kosanić, Marolt, Zrnić, Lerher (2023); Ekren (2018); Azadeh, de Koster, Roy (2019).

3. Speeds (Crane, shuttles, robots)

Malmborg (2003); Fukunari & Malmborg (2009); Ekren & Heragu (2011); Marchet, Melacini, Perotti, Tappia (2013); Lehrer, Ekren, Dukic, Rosi (2015); [AutoStore](#).

4. Cycle (single/dual)

Bozer & White (1984); Malmborg (2003); Fukunari & Malmborg (2009)

5. Depth (single/double)

Roodbergen & Vis (2009); Kosanić et al. (2023).

6. Peak Multiplier

Pazour & Meller (2014)

7. Storage (Randomized)

Kallina C, Lynn J (1976); Frazelle EH, Hackman ST, Platzman LK (1989); Gibson DR, Sharp GP (1992); Manzini, Gamberi, Regattieri (2006); Eder (2020)

Replenishment and Storage Reorganization

- **Replenishment** occasionally seizes resources to simulate restocking tasks. (Hausman et al. 1976)
- **Reorganization** reshuffles the product-aisle assignments in *self.storage_map* at regular intervals. (Heskett 1963, Kofler et al. 2014, Schuur 2015)

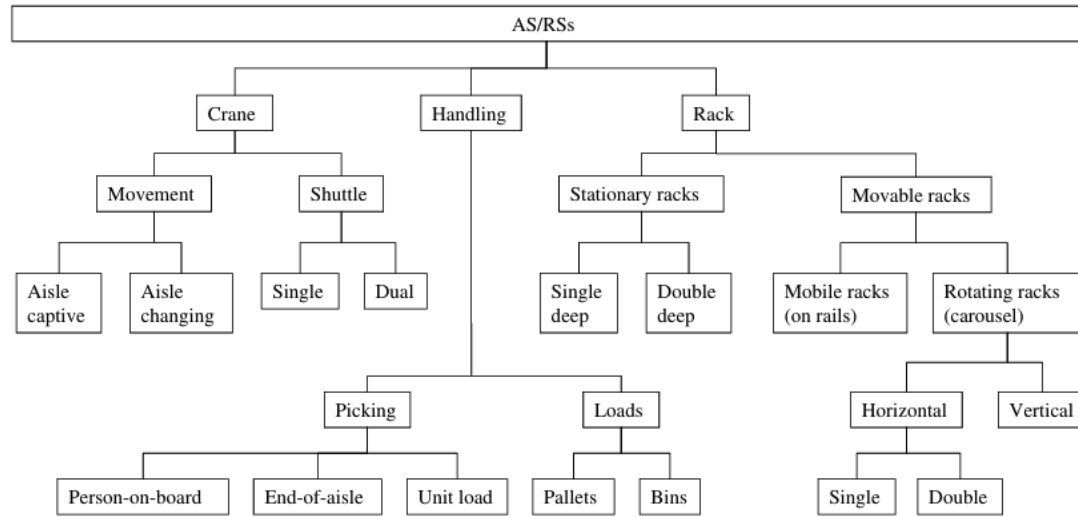


Fig. 1. Classification of various AS/RS system options.

Source: Roodbergen & Vis (2009)

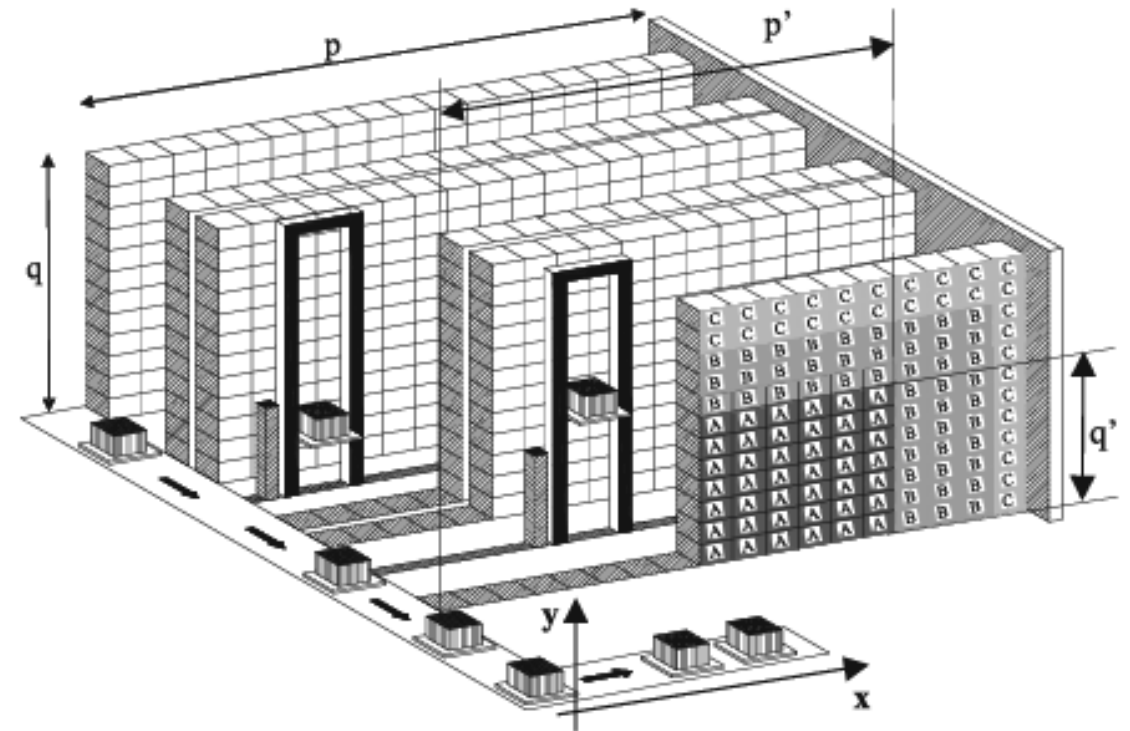


Fig. Automated storage and retrieval system with class based storage allocation. Classes A,B,C

Source: Manzini et al. (2006)

Replenishment

- Hausman, Schwarz, and Graves (1976) “investigated the cycle time calculations with ABC storage and economic order quantity–based replenishment.”
- “In a GridStore, a replenishment conveyor moves products that need to be stored in the grid.” (Gue et al. 2014)
- “... optimal replenishment level is about 50%.” (Lamballais, Roy, and De Koster 2017)
- “The replenishment is still carried out manually, but it can be done without interrupting the picking process.” (Azadeh, De Koster, and Roy 2019)
- “Robotic Mobile Fulfillment systems focuses mostly on design issues, rather than operational policies that integrate the picking, storage, and replenishment processes.” (Azadeh, De Koster, and Roy 2019)

Wave Picking

- “The traditional approach for coordinating the flow of work in warehouses with an automated sorter...,” “wave-based results rely on the key assumption that a complete wave of picked items is always available to be released into the sorter when required” Gallien and Weber (2010)
- “picking work for a given wave can only start when all the items from the previous wave have been already picked” (Choe et al. 1992, Petersen 2000)
- “...items of a given wave are only released into the sorter when all the orders from the previous wave have been already sorted or packed or both” (Armstrong et al. 1979, Meller 1997)

Discussions (contd.)

Distinct Products & Attributes; Dynamic Storage Allocation

Van den Berg and Gademann (2000) “presented one of the most complete simulation studies of AS/RS control policies. They compared several storage assignment policies as well as the sequencing of storage and retrieval requests. The simulated response used throughout the study is the trade-off between crane travel time and retrieval requests completion time.” (Gagliardi et al. 2012)

Simulation Evaluation Policies: (Van den Berg and Gademann 2000)

A. *storage location assignment* –

“The *class-based storage* policy partitions the products among a number of classes and reserves region within the rack for each class. The *randomized storage* policy allows a product to be stored anywhere in the rack. The *dedicated storage* policy assigns specific locations to each product, which may only be occupied by that product.”

B. *Request selection* rules –

longest waiting retrieval (LWR); nearest retrieval (NR); nearest storage/retrieval (NSR), as opposed to simply FCFS

C. *Open location selection* rules and *Urgency rules* –

- random open location; closest open location; nearest neighbor; shortest leg;
- prefer dual command cycle to avoid less efficient travel of the S/R machine, which might cause that the number of available storage and retrieval requests grows rapidly.

Theoretical Grounding

- 1) All storage locations are the same size as the pallets themselves. On each pallet, only one item is stacked.
- 2) The depot is at the lower-left side of the storage rack.
- 3) The storage rack is “square” in time; the time for the machine to move from the depot to the most distant column equals the time from the depot to the highest row.
- 4) The capacity of the machine is one unit load and it operates in a single-command mode; the machine either stores or retrieves a pallet each time.
- 5) The pickup/deposit time for the machine to load/unload a pallet is ignored.
- 6) The turnover of each item is known and constant through time. The turnover of an item equals the number of times the item is stored or retrieved in a unit-time period, such as a week, a month, or a year.
- 7) Item inventories are replenished according to the simple EOQ model, where the demand of every item (measured in number of pallets) is deterministic and determined by the ABC curve.
- 8) The items are ranked according to their marginal contribution to the total demand using the ABC curve. An item that has a smaller contribution is indexed with a larger number.

Source: Hausman, Schwarz, Graves (1976)

Hausman, W. H., Schwarz, L. B., & Graves, S. C. (1976). Optimal storage assignment in automatic warehousing systems. *Management science*, 22(6), 629–638

Also adapted by Gagliardi et al. (2012) and Yu, de Koster, Guo (2015).

Theoretical Grounding (contd.)

Source:

Gagliardi, J. P., Renaud, J., & Ruiz, A. (2012). Models for automated storage and retrieval systems: a literature review. *International Journal of Production Research*, 50(24), 7110–7125.

Travel-time models give a good approximation of crane travel times under specific conditions. This approximation is of interest because the **AS/RS throughput is inversely proportional to the crane travel time**. (Gagliardi, Renaud, Ruiz 2012)

Is this the case in the simulation?

Not Necessarily.

Here's why:

- i. Throughput Depends on Multiple Factors
- ii. Queueing Dynamics
- iii. Empirical Relationship

Can we mathematically map the relationship over here based on the simulation?

Not in closed form, but empirically.

What This Simulation Can Do For You:

1. Compare performance across different automated storage systems
2. Predict throughput, costs, and fulfillment times before physical implementation
3. Test changes virtually before applying them in your facility
4. Quantify the ROI of different warehouse AS/RS technologies
5. Optimize storage policies based on your product movement patterns

Advanced Features

- Dynamic storage reassignment simulation
- Wave and batch picking optimization
- Resource utilization analysis
- Peak demand simulation
- Real-time dashboard for ongoing optimization
- SKU velocity profiling



How to Get Started?

- **Input your current system**

- ☐ Storage system type
- ☐ Number of resources (cranes, shuttles, robots)
- ☐ Number of Aisles

- **Set your product flow parameters**

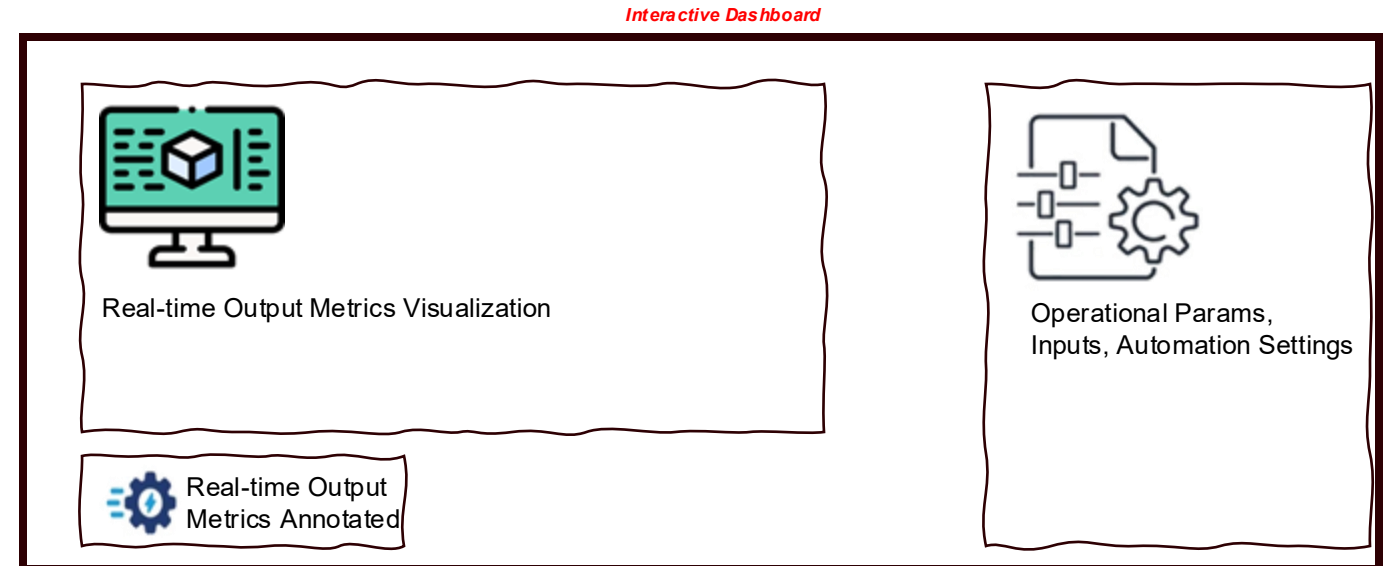
- ☐ SKU distribution
- ☐ ABC storage policy
- ☐ Batch processing preferences

- **Run simulations and compare scenarios**

- ☐ Test what-if scenarios
- ☐ Compare multiple storage policies
- ☐ Evaluate different equipment configurations

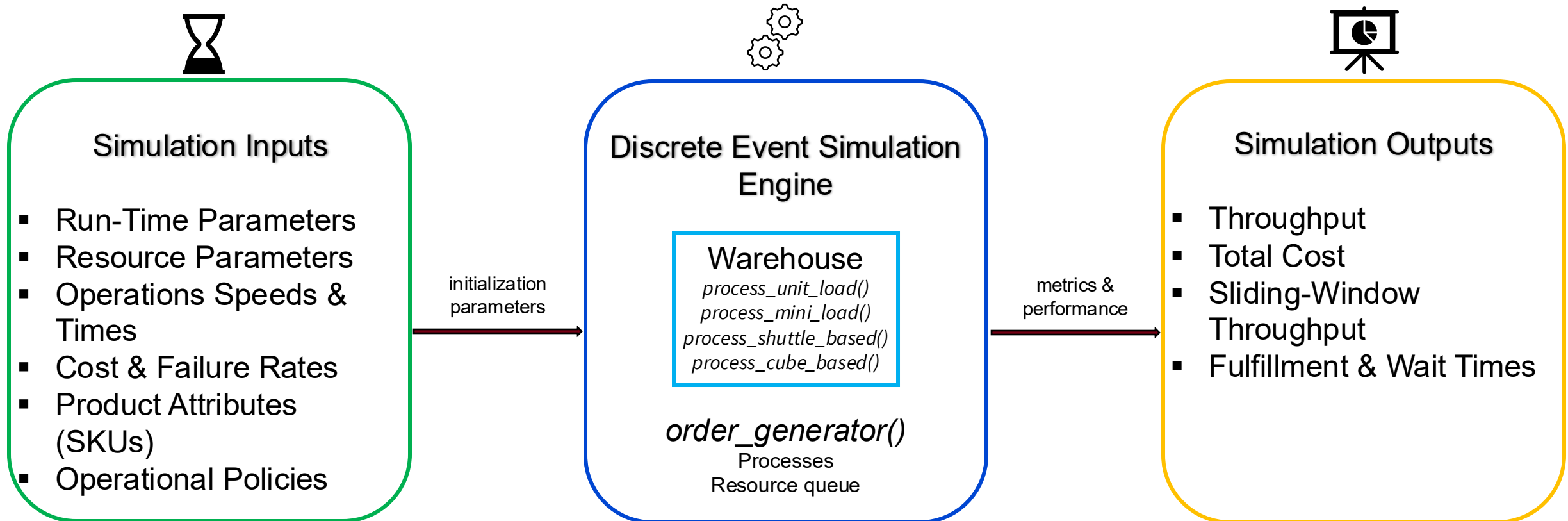
- **Implement with confidence**

- ☐ Data-backed decisions
- ☐ Predictable outcomes
- ☐ Measurable ROI



Simulation Overview

1.  Simulation Architecture
2. Simulation Features
 - a.  Simulation Inputs
 - b.  Simulation Outputs with visualization



Data Required (Parameters & Inputs)

System Configuration Data

1. Current AS/RS storage system type(s) in use:

- Unit-load; Miniload; Shuttle-based; Cube-based
- Any hybrid configuration

2. Equipment quantities:

- Number of cranes; Number of shuttles; Number of robots
- Number of aisles

3. Storage configuration:

- Single-deep or double-deep storage
- Single command or dual command cycles
- Static or dynamic storage allocation strategy

Financial Data - Cost information

- Automated system operating costs
- Cross-docking operation costs (if applicable)
- Labor costs (if relevant)

Operational Data

1. Order patterns:

- Average orders per hour (baseline)
- Peak order rates (orders per hour during busiest periods)
- Peak multiplier (ratio of peak to average)
- Seasonal patterns (if applicable)

2. Order characteristics:

- Average number of items per order
- Product mix percentages (type A, B, C distribution)
- Batch picking data (if applicable)
- Wave picking schedules (if applicable)

3. Equipment performance metrics:

- Crane vertical and horizontal speeds, Shuttle speeds
- Acceleration/deceleration times
- Equipment failure rates; Average downtime during failures

4. Processing times:

- Average retrieval times by product type
- Cross-docking rates and times
- Replenishment frequency

Performance Benchmarks - Current KPIs

- Current average throughput (orders/hour)
- Average order fulfillment times
- Average wait times; Queue lengths
- Cost per order

System Configuration

1. System Selection

Select the AS/RS system that defines your operation (from the bottom buttons).

2. Resource Selection

Determine the number of resources (cranes/shuttles/robots) being used in your operation. The configuration for unit-, mini-, and shuttle-based systems are aisle captive. For cube-based, if aisles= n , then the system has an $n \times n$ grid.

☐ Design & Resource Optimization

After selecting your preferred system type, vary the number of aisles and number of resources using the slider. Then, observe the impact on performance.

3. Peak Demand Planning

Change the peak multiplier to understand how your system can handle peak demand periods. For example, peak multiplier of 1.4 implies a 40% higher demand.

4. Command Cycle and Storage System

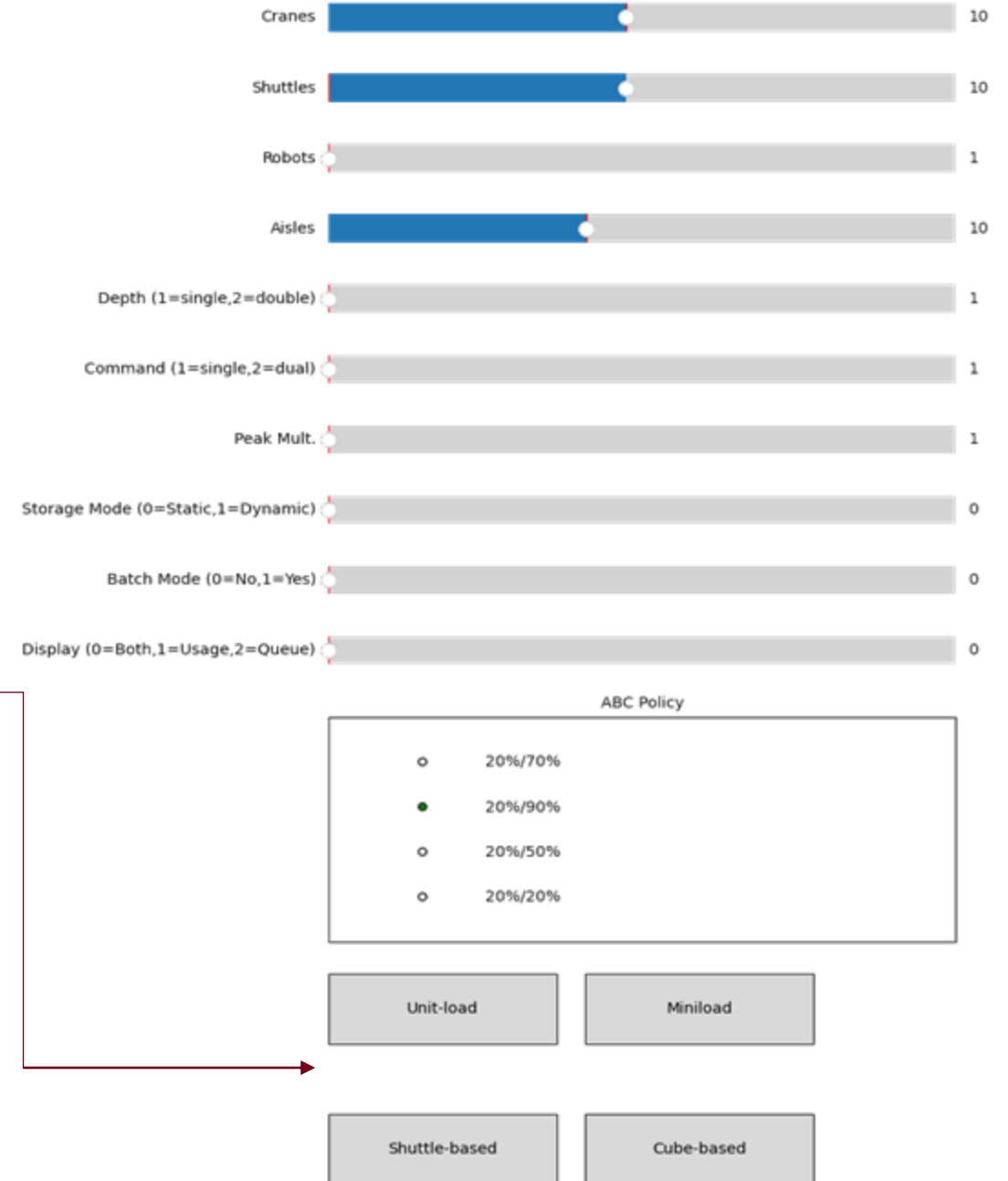
Select the retrieval command (single/dual) and storage system (single/double deep) based on your system.

5. Batching Strategy Evaluation

Select a picking strategy (Single-order, Batch, Wave) to evaluate the system efficiency.

6. Storage Policy Evaluation

Select a storage policy to evaluate the system performance.



The interface displays various configuration sliders and buttons. A red line connects the 'System Selection' step to the bottom buttons, and another red line connects the 'Design & Resource Optimization' step to the sliders.

Parameter	Value
Cranes	10
Shuttles	10
Robots	1
Aisles	10
Depth (1=single,2=double)	1
Command (1=single,2=dual)	1
Peak Mult.	1
Storage Mode (0=Static,1=Dynamic)	0
Batch Mode (0=No,1=Yes)	0
Display (0=Both,1=Usage,2=Queue)	0

ABC Policy

- ☐ 20%/70%
- ☒ 20%/90%
- ☐ 20%/50%
- ☐ 20%/20%

Unit-load Miniload

Shuttle-based Cube-based

Operational & Financial Costs Data

1. Order Patterns

- **Average orders per hour (baseline):**
 - ☐ First 4 hours: 200 orders/hr; After 4 hours: 150 orders/hr
- **Peak multiplier:**
 - ☐ Default value: 1.0 (can be adjusted via slider from 1.0 to 3.0)

2. Order Characteristics

- **Average number of items per order:**
 - ☐ Random between 2-4 items per order
- **Batch picking data:**
 - ☐ 38% of batches are size 10 (maximum); Remainder are size 3 (45% of non-maximum) or size 4 (55% of non-maximum)
 - ☐ Maximum items capacity per batch: 35
- **Wave picking schedules:**
 - ☐ Wave interval: 120 seconds (2 minutes)

3. Equipment Performance Metrics

- **Crane speeds:**
 - ☐ Vertical speed: 1.0 units/s; Horizontal speed: 1.0 units/s
 - ☐ Acceleration/deceleration time: 1.2 s
- **Shuttle speeds:**
 - ☐ Speed: 3.0 units/s
 - ☐ Acceleration/deceleration time: 0.8 s
- **Equipment failure rates:**
 - ☐ Base failure rate: 0.01 (1%);
 - ☐ Unit-load systems: 0.015 (1.5%); Miniload systems: 0.008 (0.8%)
- **Average downtime during failures:**
 - ☐ Unit-load: 20-40 seconds; Miniload: 5-15 seconds
 - ☐ Other systems: 10-30 seconds

4. Processing Times

- **Average retrieval times by product type (base multipliers):**
 - ☐ Type A: 1.0 (base processing time); Type B: 1.2 (20% longer than A); Type C: 1.5 (50% longer than A)
- **Travel factors by product type:**
 - ☐ Type A: 1.0 (base travel distance); Type B: 1.1 (10% longer travel than A); Type C: 1.3 (30% longer travel than A)
- **Cross-docking rates and times:**
 - ☐ Default rate: 0.0
 - ☐ Processing time: Random between 2-4 seconds, multiplied by product processing multiplier (If default rate > 0)
- **Replenishment frequency:**
 - ☐ Every 1800 seconds (30 minutes)
 - ☐ Duration: Random between 5-10 seconds per resource

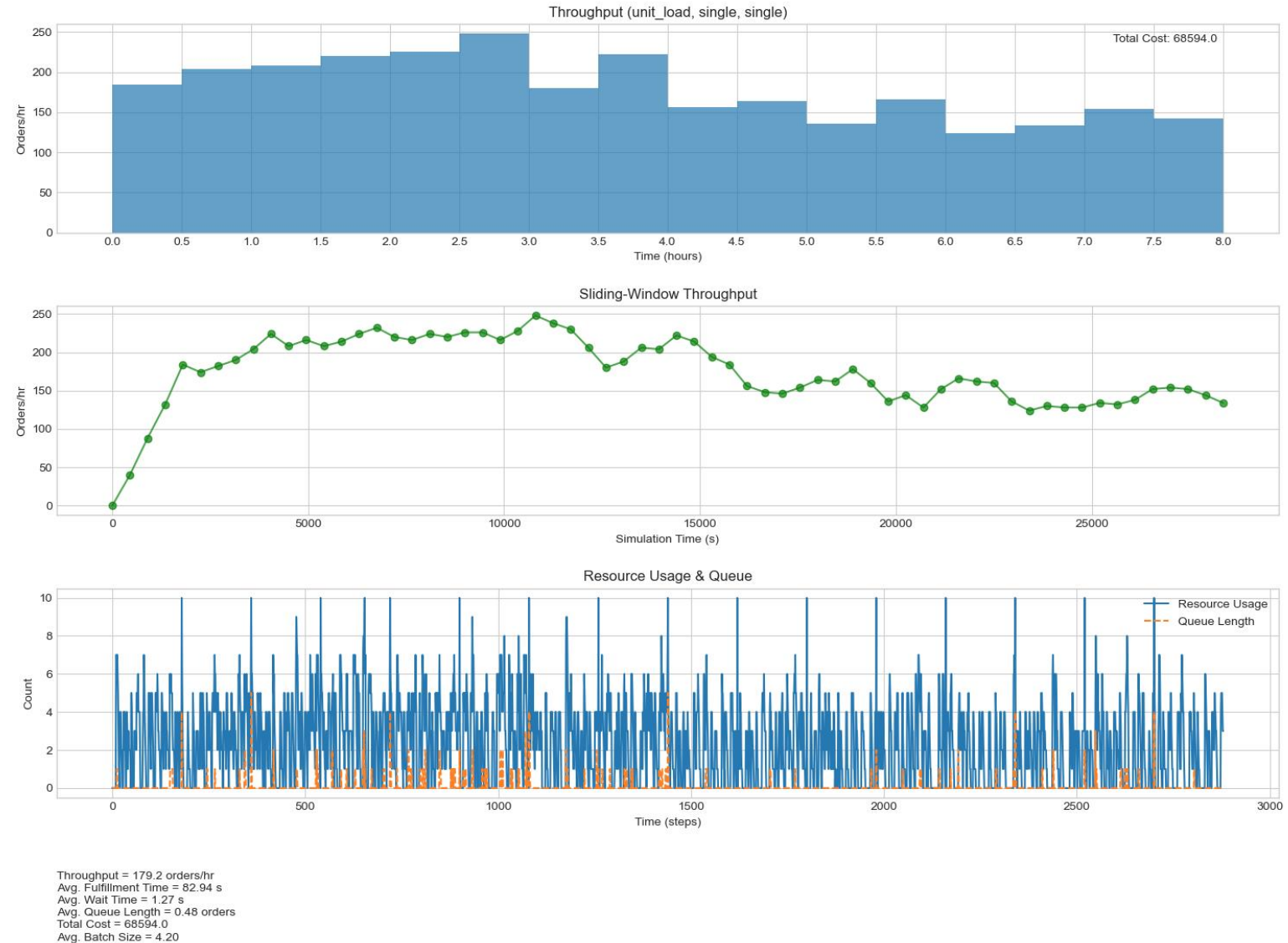
5. Financial Data - Cost Information

- **Automated system operating costs:**
 - ☐ Base automated cost rate: 1.0 units/s
 - ☐ This applies to all time periods when resources (cranes, shuttles, robots) are being used
- **Cross-docking operation costs:**
 - ☐ Cross-docking cost rate: 0.8 units/s
 - ☐ This is 20% lower than the automated system costs
 - ☐ Applied during cross-docking operations, if applicable.
- **Labor costs:**
 - ☐ Not explicitly modeled in the current simulation
 - ☐ Could be incorporated as an extension by adding labor requirements for certain operations

These values are encoded in the simulation parameters and behavior, though they may be modified through the interactive dashboard for experimentation.

Key Metrics/Outputs

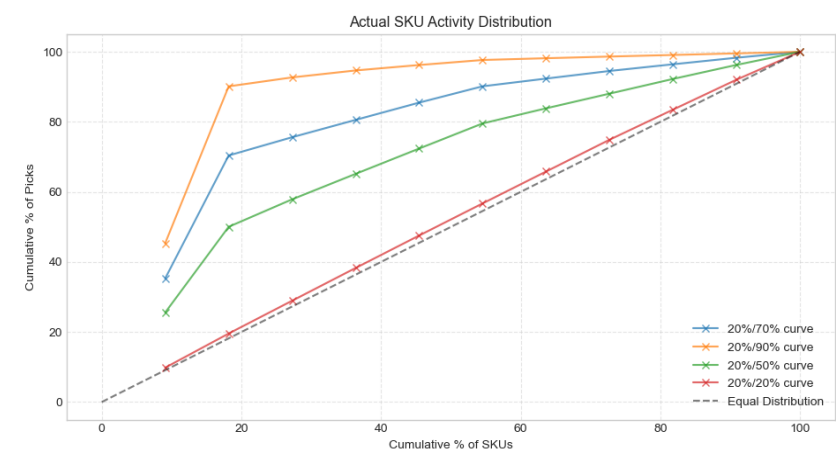
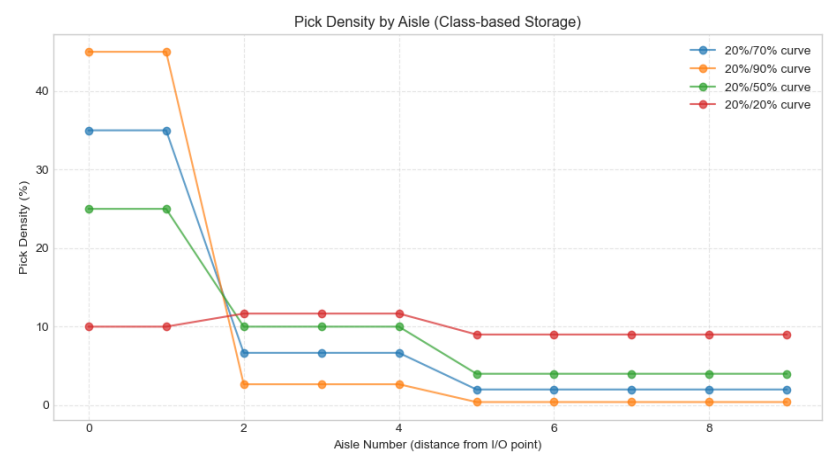
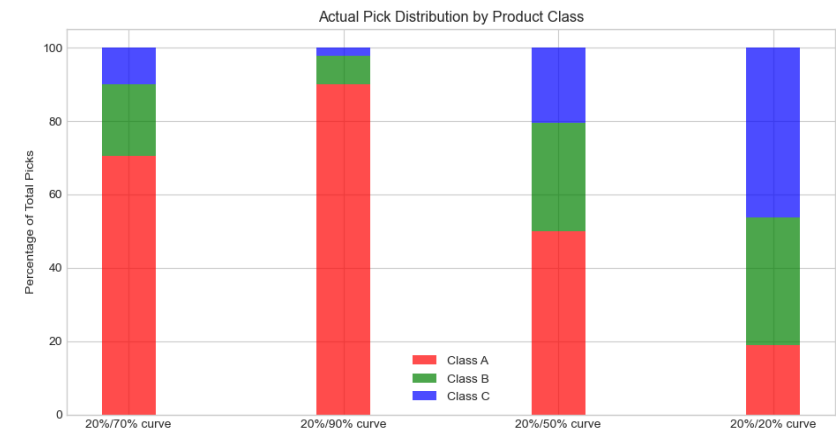
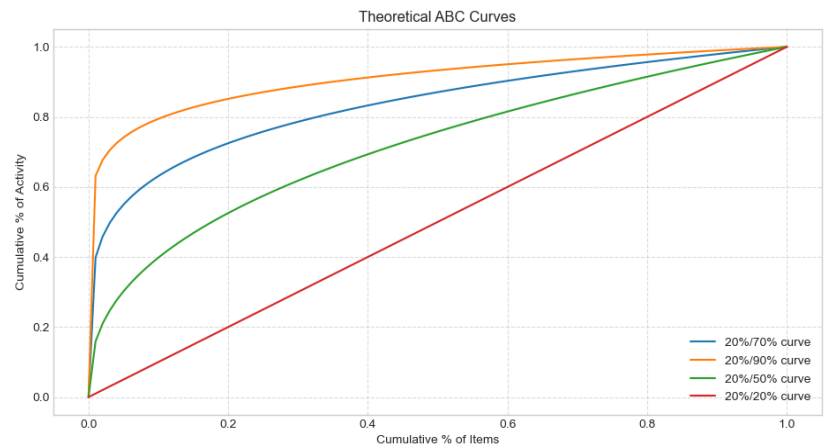
- Throughput:** Orders processed per hour.
Lit: Malmborg (2002); Marchet et al. (2013); Zaerpour, Volbeda & Gharehgozli (2019).
- Sliding-window Throughput:** 30-minute-window throughput rate every computed every 7½ minutes.
- Avg. Fulfillment Time:** Average time from order arrival to completion.
- Avg. Wait Time:** Average time orders spend waiting for resources.
- Avg. Queue Length:** Average number of orders waiting for processing.
Lit: Daniel Muller (1989); Malmborg (2002); Riccardo Manzini, Mauro Gamberi, Alberto Regattieri (2006); Gu, Goetschalckx, McGinnis (2010); Roy, Krishnamurthy, Heragu, Malmborg (2012); Roy, Krishnamurthy, Heragu, Malmborg (2017)
- Total Cost:** Cumulative operational cost.
Lit: Pazour & Meller (2014); Schäfer, Lorson, Hübner (2025)
- Avg. Batch Size:** Average number of orders in each batch (when batching is enabled).



Storage Policies: ABC Curves

Can be used for-

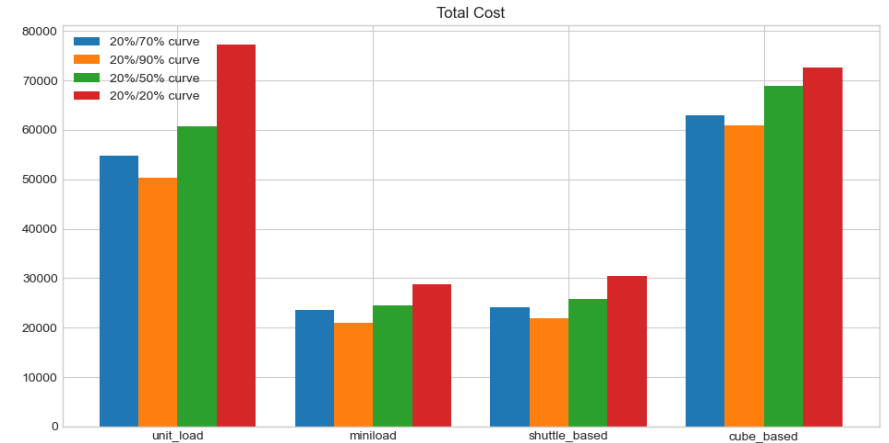
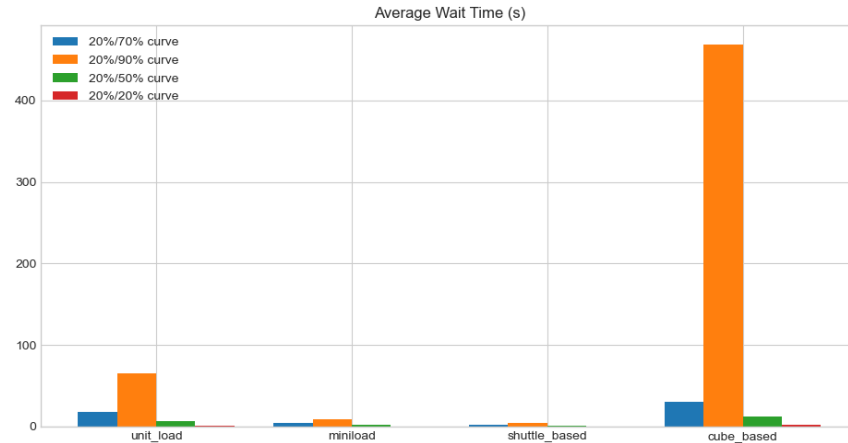
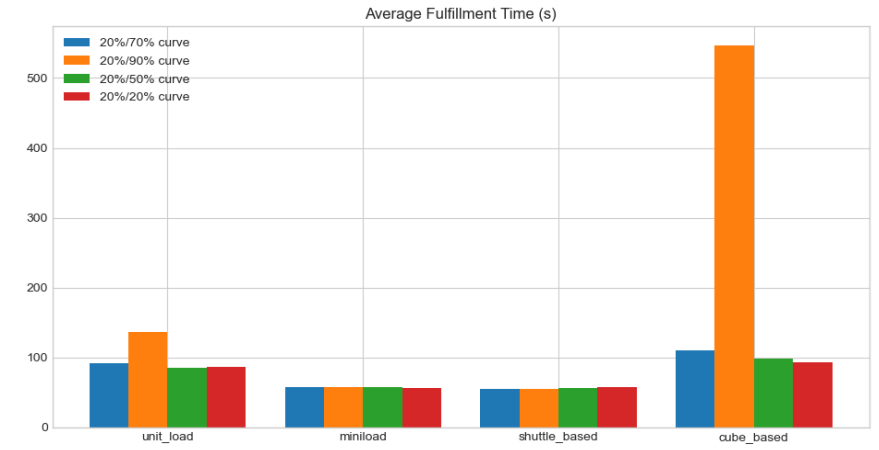
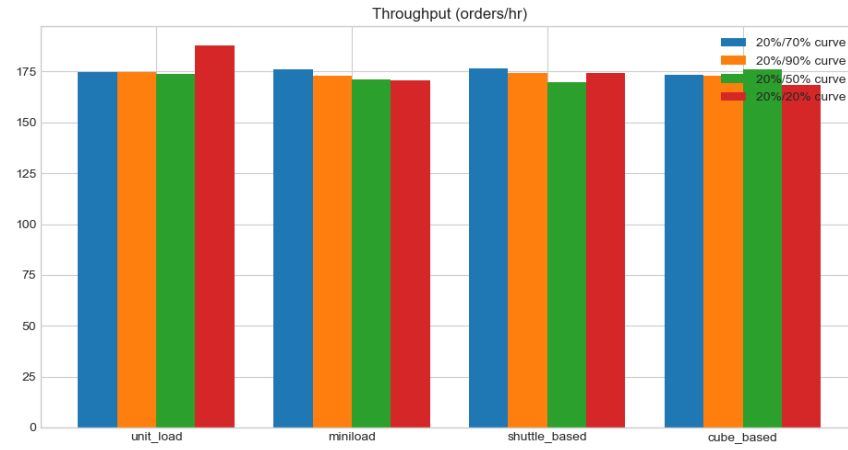
- Theoretical vs. Actual
- Policy Selection Guidance
- Visualization



Storage Policies: Performance Metrics

Can be used for-

- Throughput Comparison
- Fulfillment Time Analysis
- Wait Time Revelation
- Cost Efficiency

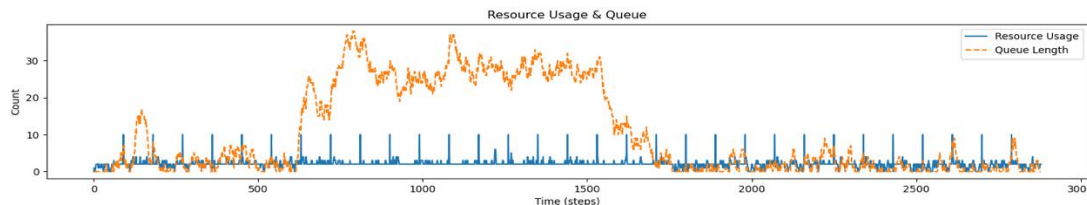


Cube-based AS/RS: Metrics Interpretation

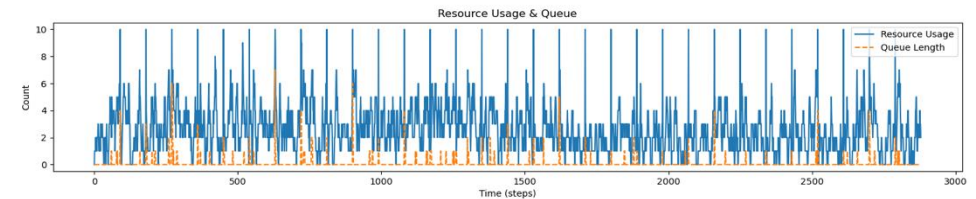
Storage Policy	Total Cost (cost units)	Avg. Fulfilment Time (in s)	Avg. Wait Time (in s)	A/B/C Class Distribution (in %)
20%/70%	62639	69.14	23.29	69.2/20.7/10.1
20%/90%	58760	262.74	218.85	90.1/7.6/2.2
20%/50%	68063	56.42	7.05	49.5/30.4/20.1
20%/20%	74241	55.55	0.80	19.3/34.7/46

This policy represents a balanced approach with moderate concentration of activity. The class distribution shows that roughly 70% of picks come from A-class items (which comprise 20% of SKUs). The fulfillment and wait times are moderate, and the total cost is second-lowest among the four policies. This represents a good balance between efficiency and operational stability.

This policy shows the most extreme concentration, with over 90% of activity focused on just 20% of SKUs. While it achieves the lowest total cost (suggesting operational efficiency), it comes with a severe penalty in fulfillment time (nearly 4x higher than other policies) and wait time (30–200x higher). This indicates severe bottlenecks and congestion around the A-class storage locations. The robots are likely spending significant time waiting for access to the same small portion of the storage grid.



The equal distribution policy shows the highest cost but delivers the best performance in terms of fulfillment time and nearly eliminates wait time entirely. With activity distributed almost equally across all product classes (slightly skewed toward C-class items), there's minimal congestion and very little robot waiting time. The class distribution confirms that this policy effectively treats all products nearly equally, with activity roughly proportional to the number of SKUs in each class.



Higher queue for 20/90 policy compared to 20/20.

Cube-based AS/RS: Metrics Interpretation (contd.)

In cube-based storage (like AutoStore or similar grid-based systems), the impact of I/O point location is particularly significant due to the unique movement patterns.

For example, when implementing a 20%/90% policy in a cube-based system, we're essentially saying a small number of products account for almost all our activity. If the I/O points are positioned ideally near the A class items:

- Robots travel shorter distances for most retrievals
- Popular products require fewer moves to access
- Less overall system movement occurs for the same throughput

However, if I/O points are poorly positioned relative to high-activity products, the exact opposite happens; robots must constantly travel across the entire grid, making many puzzle moves to access frequent items. The simulation's high cost for 20%/90% policy with cube-based systems suggests the model assumes standard placement rather than optimized I/O positions.

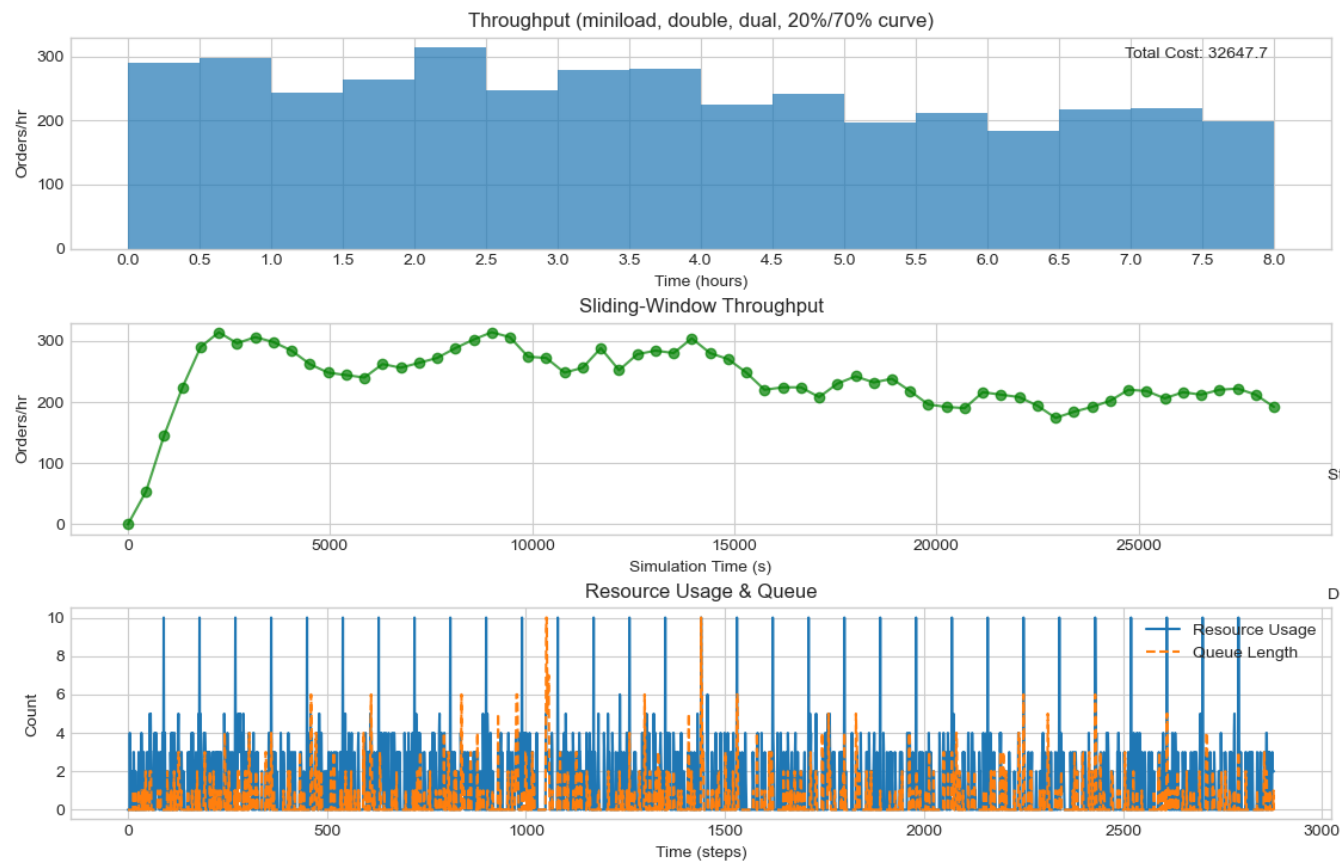
Business Implications of Simulation Results

1. **Layout Design Opportunity:** For new installations, these results suggest that careful consideration of I/O point placement relative to product velocity could significantly reduce operational costs.
2. **Policy-Layout Alignment:** The optimal storage policy depends heavily on I/O configuration. A facility with edge-based I/O points might perform best with a 20/90 policy, while a facility with centralized I/O might benefit more from a 20/50 policy.
3. **Dynamic I/O Assignment:** Advanced systems could even dynamically assign different robots to different I/O points based on the product classes they're handling.
4. **Diminishing Returns:** The data shows that I/O optimization provides diminishing returns as policies become more distributed. For operations that need predictable fulfillment times, a more distributed policy with less I/O sensitivity might be preferable.

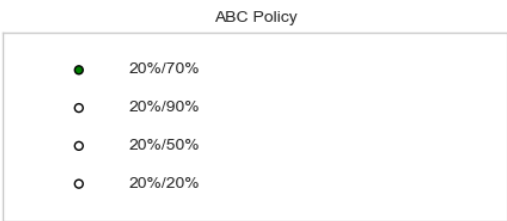
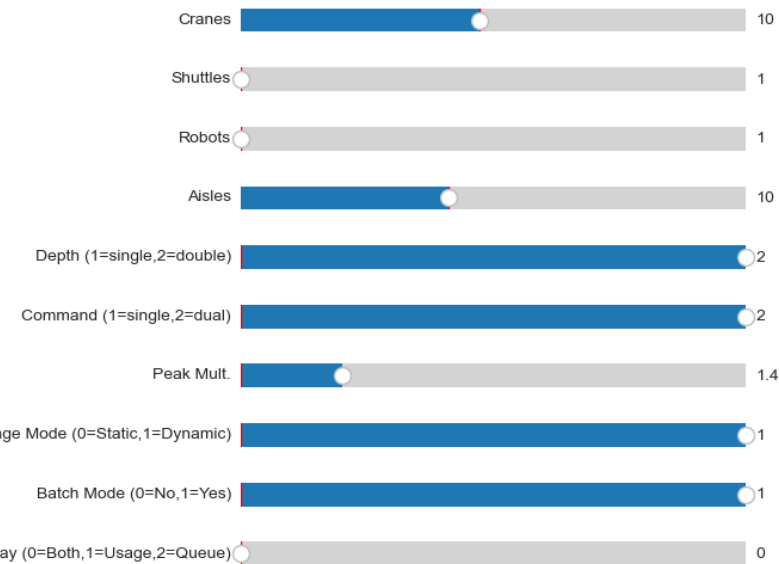
These considerations can also account for the following tradeoffs:

- **Cost vs. Speed Tradeoff:** The results show a clear inverse relationship between cost and fulfillment speed.
- **Congestion Effects:** The extreme wait time in the 20/90 policy (218.85 seconds) reveals how cube-based systems are particularly susceptible to congestion when activity is highly concentrated. This is likely due to the puzzle-movement nature of these systems, where multiple retrievals from the same area create gridlock.
- **Optimal Balance Point:** The 20/50 policy appears to offer a compelling balance point - it's only 8.7% more expensive than the 20/70 policy but delivers 18% faster fulfillment and 70% less waiting time.
- **Policy Selection Factors:** For operations where minimizing cost is the primary goal, the 20/90 policy might be attractive despite its performance issues. For operations where consistent, predictable fulfillment times are critical (ex., e-commerce with same-day delivery), the 20/50 or even 20/20 policies would be preferable.
- **Grid Utilization:** In cube-based systems, the 20/20 policy makes better use of the entire storage grid, while the 20/90 policy leaves large portions of the grid underutilized while creating intense competition for access to A-class locations.

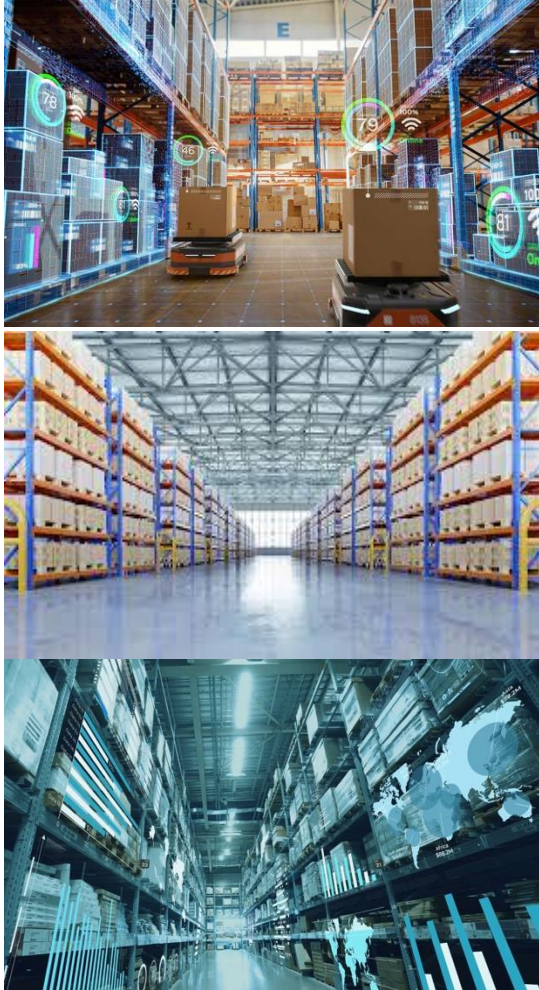
Dashboard with Storage Policy Options



Throughput = 244.4 orders/hr
Avg. Fulfillment Time = 47.67 s
Avg. Wait Time = 5.47 s
Avg. Queue Length = 2.02 orders
Total Cost = 32647.7
Storage Policy: 20%/70% curve
Class Distribution: A=70.8%, B=19.6%, C=9.6%
Avg. Batch Size = 4.15



Business Value



1. Concrete Business Value

- Higher throughput with optimized storage policies
- Improved fulfillment times by avoiding congestion
- Better expenditure decisions with virtual testing

2. Risk Reduction

- Identify potential bottlenecks before they occur
- Validate storage policy changes before implementation
- Quantify performance impacts before making investments

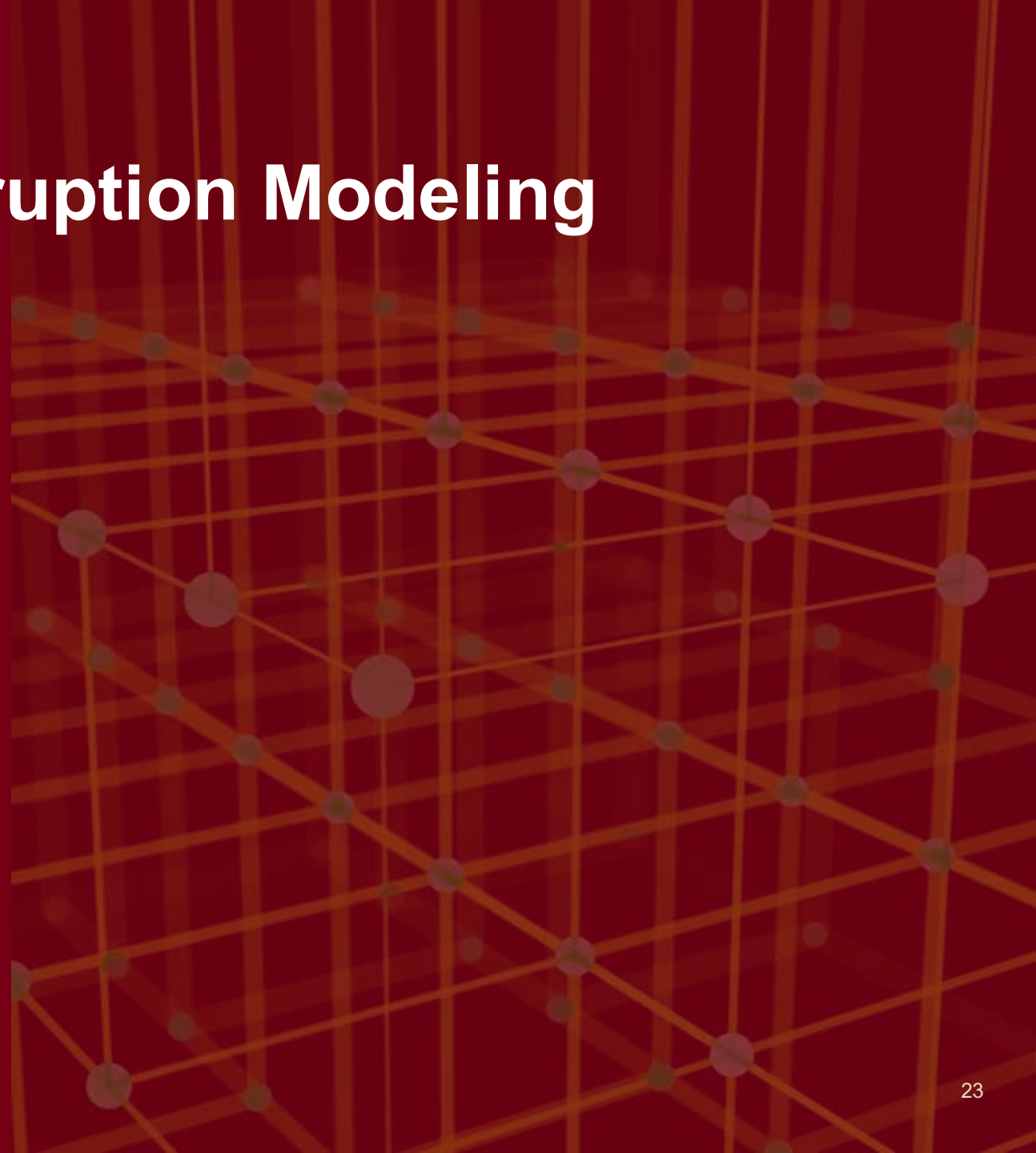
3. Visualization Capability

- Performance metrics quantify the impact of different policies
- ABC curves show theoretical vs. actual performance
- Pick density charts reveal potential congestion points

4. Adaptability

- Test various storage system types (unit-load, miniload, shuttle-based, cube-based)
- Evaluate different resource allocations (cranes, shuttles, robots)
- Compare multiple ABC storage policies (20%/70%, 20%/90%, etc.)

Automated Warehouse – Policy Evaluation and Disruption Modeling



Model Recap

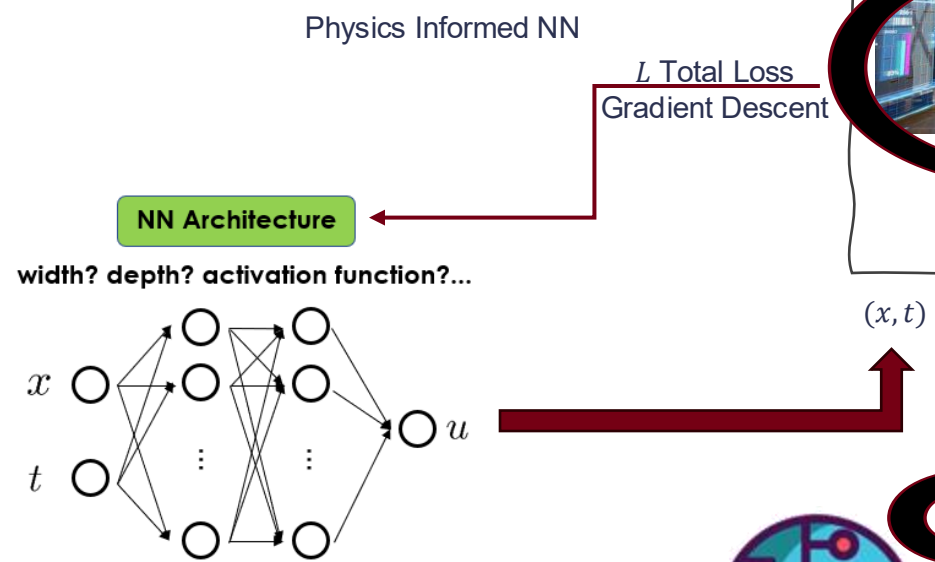
The conceptual model considers:

1. Queuing dynamics for resource allocation (FIFO queues).
2. Random inter-arrival times (Poisson process).
3. Realistic assumptions of equipment speed and storage allocation strategies (randomized, class-based, dedicated).
4. Potential operational interruptions (ex., replenishment, product reshuffling, batch picking) to realistically reflect warehouse operations.

How to improve the academic rigor?

- How to evaluate the simulation w/o real-world warehouse data?

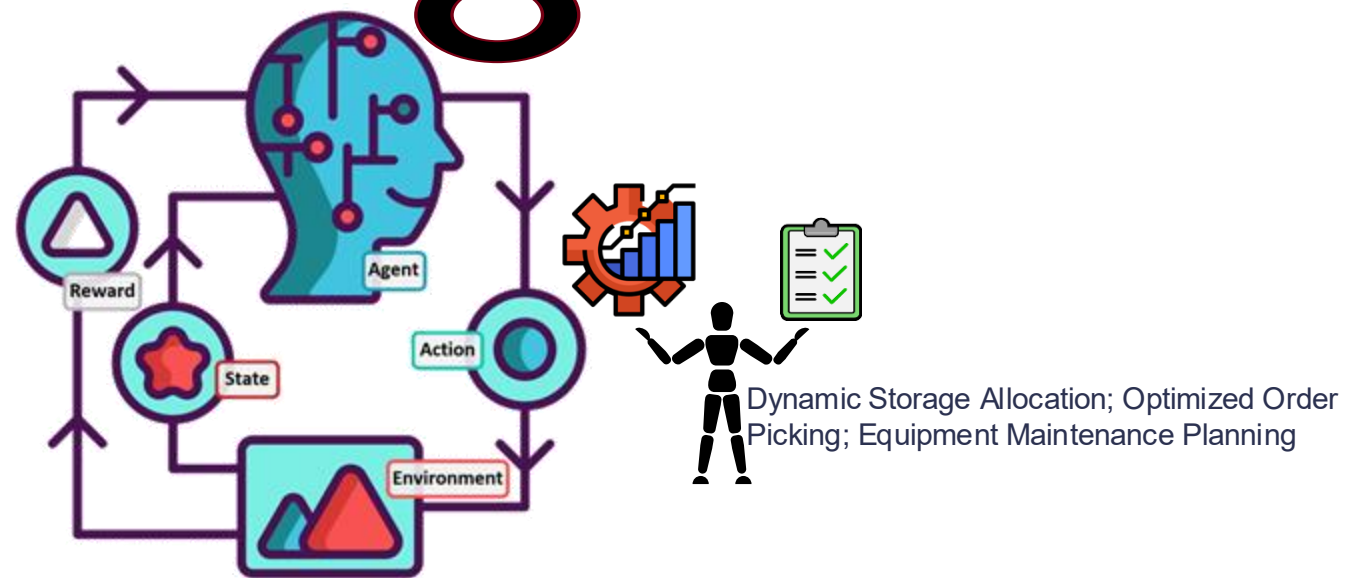
Automated Warehouse Simulation: RL-based Resilience Technology Selection and Disruption Modeling



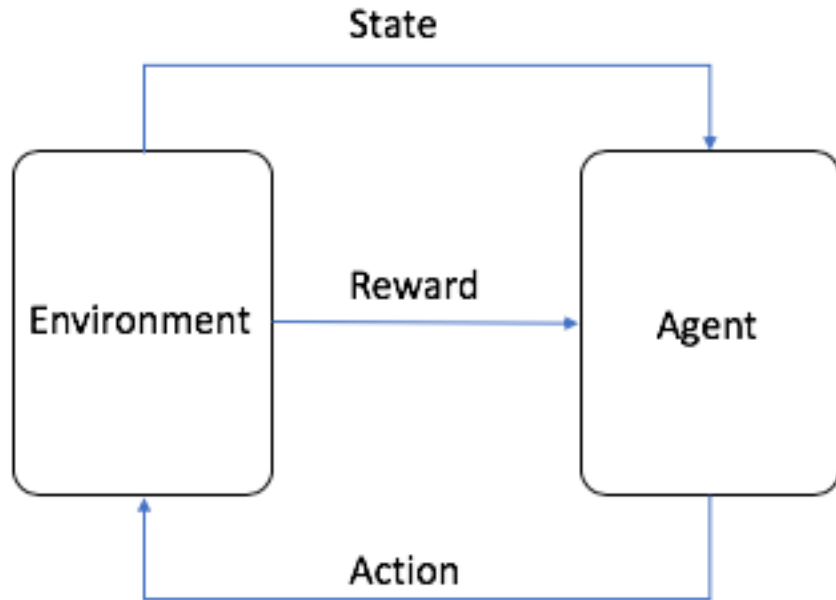
Automated Warehouse Digital Twin

Discrete Event Simulation:
AS/RS Technology Simulator; Warehouse Capacity
Planning; Operational Resilience Analyzer

Simulation Domain



RL: A Primer



“RL is the study of agents and how they learn by trial and error. It formalizes the idea that rewarding or punishing an agent for its behavior makes it more likely to repeat or forego that behavior in the future.”

“RL is focused on goal-directed learning from interaction” (Barto & Sutton 1998)

RL Component Mapping in a dynamic warehouse simulation:

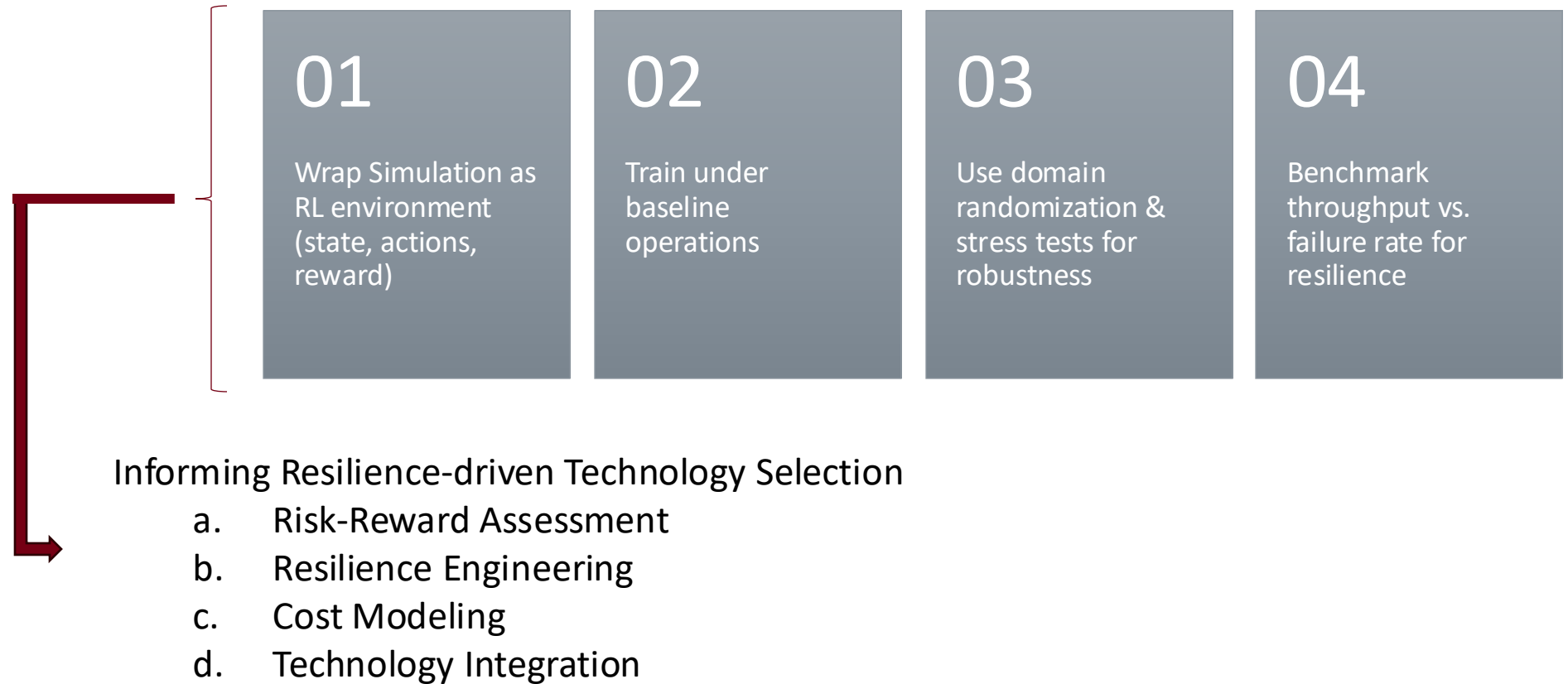
1. **Agent:** The RL (PPO) Algorithm
2. **Environment:** The simulated warehouse
3. **State:** current demand, resource status, aisle assignments, reorganization/replenishment/maintenance windows etc.
4. **Action:** Control decisions (ex., swapping/reallocating, which crane handles next request, when to release order batch etc.)
5. **Reward:** A numerical score (ex., -ve (Total Cost), throughput-based, or a combination of multiples)

Define a function ***multi_objective_cost()*** that combines:

- Cost,
- Average Wait Time in the last interval,
- Inverse Throughput (to encourage higher throughput).

The reward is computed as $-(\text{new_cost} - \text{old_cost})$, so if the new cost is lower, the agent gets a positive reward. This logic is used inside *reorganization_process* whenever the PPO agent reassigns aisles.

RL for Policy Evaluation & Disruption Modeling



Proximal Policy Optimization

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

```
class PPOAgent(nn.Module):  
    def forward():  
        ...  
  
    def choose_action():  
        ...  
  
    def store_transition():  
        ...  
  
    def update():  
        ...
```

The **constructor** initializes the policy and value networks with a chosen hidden dimension, sets up an optimizer, and creates a memory buffer to store experiences.

The **forward** method receives a batch of states, passes them through either a feed-forward or LSTM network, and outputs the policy logits and a state value estimate.

The **choose_action** method uses these logits to sample an action from a categorical distribution, returning both the action and the log probability for training.

The **store_transition** method appends each experience tuple (state, action, log probability, reward, next state, done, value) to the agent's memory.

The **update** method performs the PPO optimization loop by calculating discounted returns, normalizing them, and applying the clipped policy gradient objective across several epochs to improve the policy and value networks.

Next Step: RL-based Disruption Modeling

Adding to the existing simulation:

1. Expand the Environment

Add failure events (random or scheduled).

2. Extend the State

Include resource status flags, queue lengths, repair timers, etc.

3. Reward Shaping

Impose penalties for unfilled orders or long queues during failures. Possibly reward the agent for quick reassignments that minimize downtime impact.

4. Train & Evaluate

Run multiple episodes with disruptions at different times. Compare RL's performance (throughput, cost, waiting and recovery time) to simpler or static approaches.

